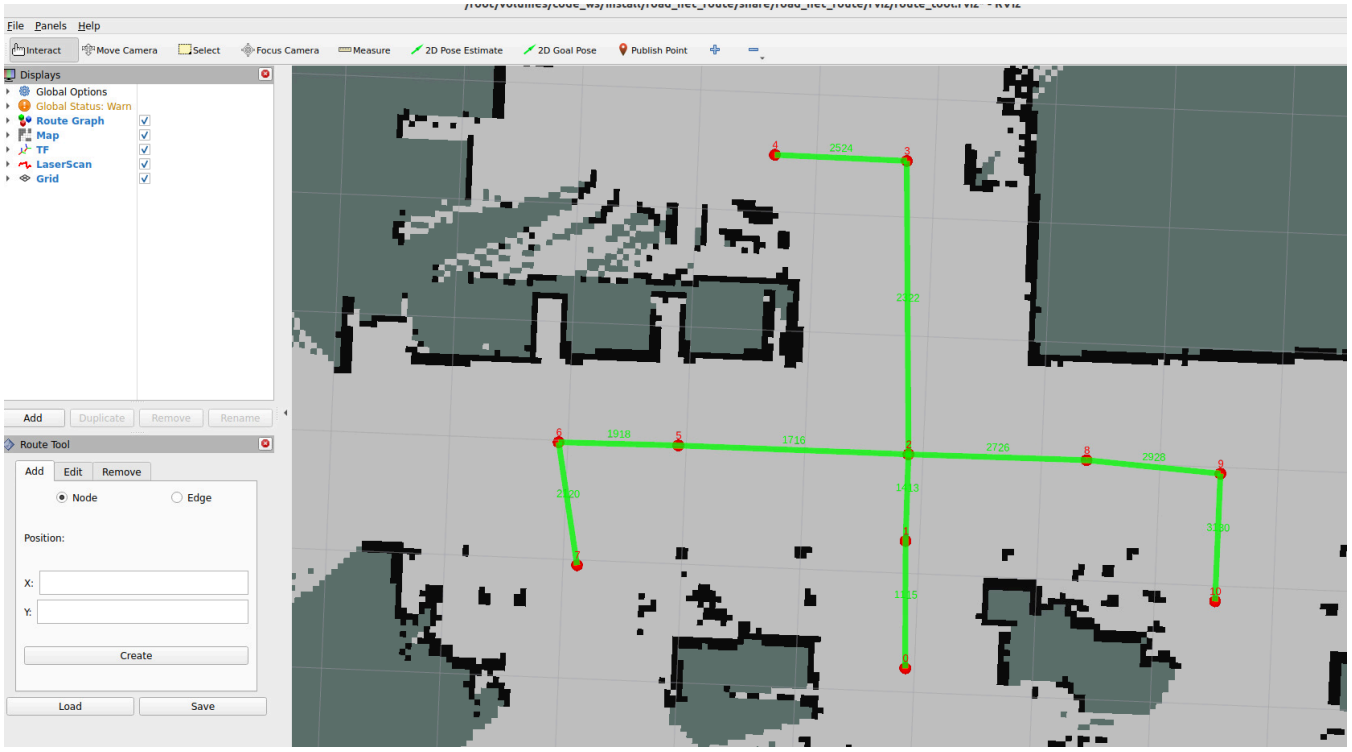
Path Replanning

1. Course Content

> **Important**
>
> - Basics
>
> Run the example program. Based on road network planning and navigation, if obstacles are detected blocking the planned path during travel, it will enter replanning mode to find a new path in the road network.
>
> - Advanced
>
> 1. Understand the implementation method of replanning and how to modify replanning strategies (strategies are not unique and can be implemented according to requirements)
>
> 2. Debug and modify obstacle detection thresholds based on queried cost values

2. Edit Road Network

- To demonstrate the replanning functionality, we add some edges to a new road network. For instructions on opening and editing road networks, refer to: **03-Road Network Annotation**, Section 4: Loading and Modifying Road Network Description

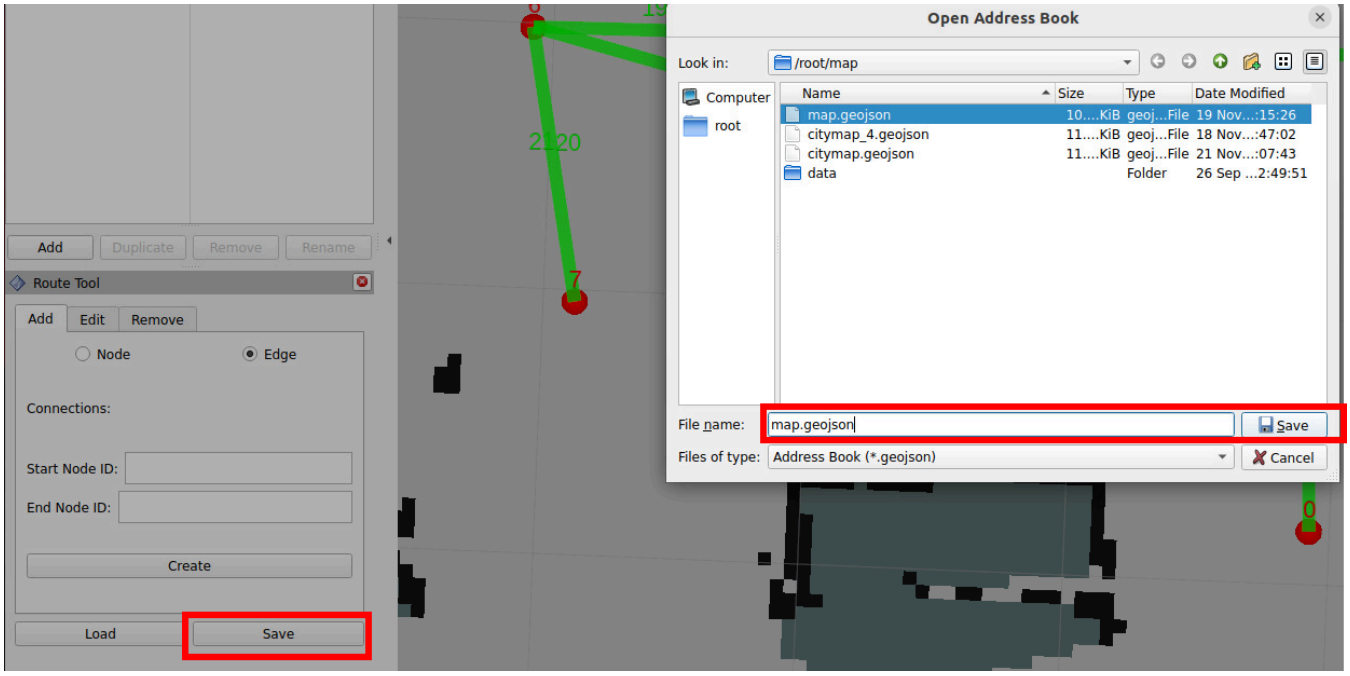


- We add bidirectional edges between waypoint 1 and waypoint 6

- All operations will have corresponding records in the terminal

```
[rviz2-1] [INFO] [1764245498.392310926] [route_tool_node]: Adding edge from 1 to 6
[rviz2-1] [INFO] [1764245596.474192853] [route_tool_node]: Adding edge from 6 to 1
```

- Then save the file



2. Run Replanning Example

2.1 Launch Road Network Planning Navigation Node

- (Host side) Chassis LiDAR driver

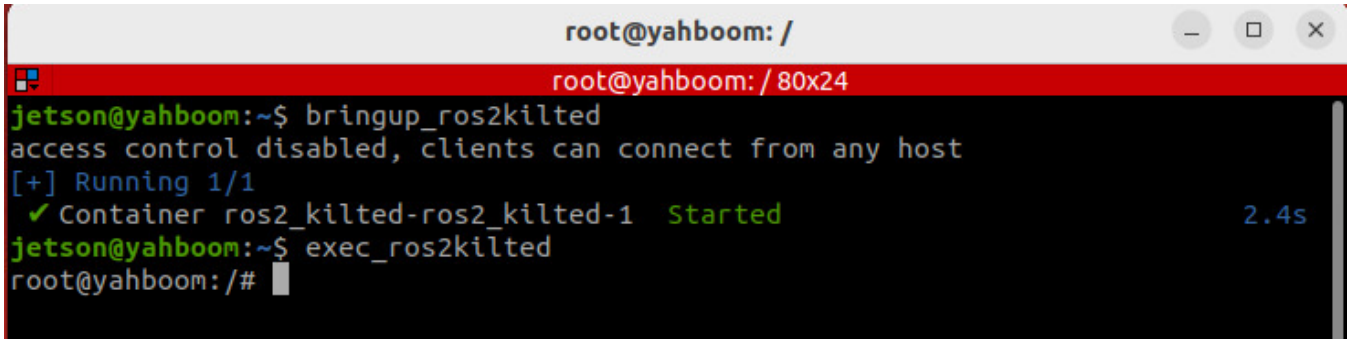
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

- （虚拟机端）启动可视化

```
ros2 launch road_net_route rviz_only.launch.py
```

- (Host side) Start the visualization and open a terminal inside a container.

```
# If the ros2_kilted docker container is not started, you need to start the container first
bringup_ros2kilted
# Enter the container
exec_ros2kilted
```



Launch road network planning navigation node

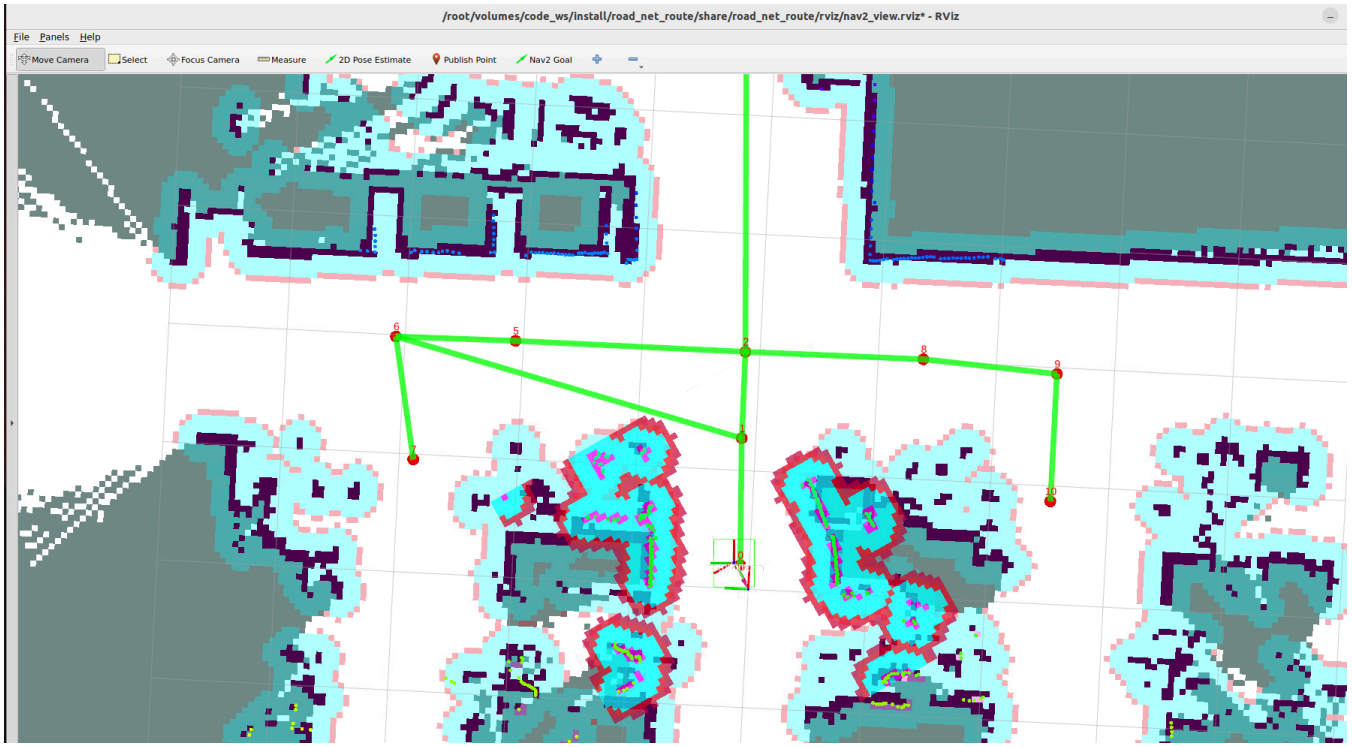
```
ros2 launch road_net_route roadnet_nav2.launch.py map:=map.yaml pose_map:=map
roadnet_file:=/root/map/map.geojson
```

- (Host side) Start **cartograph** for fast localization

```
#orin
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/map.pbstream
#rdk
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/sunrise/map/map.pbstream
```

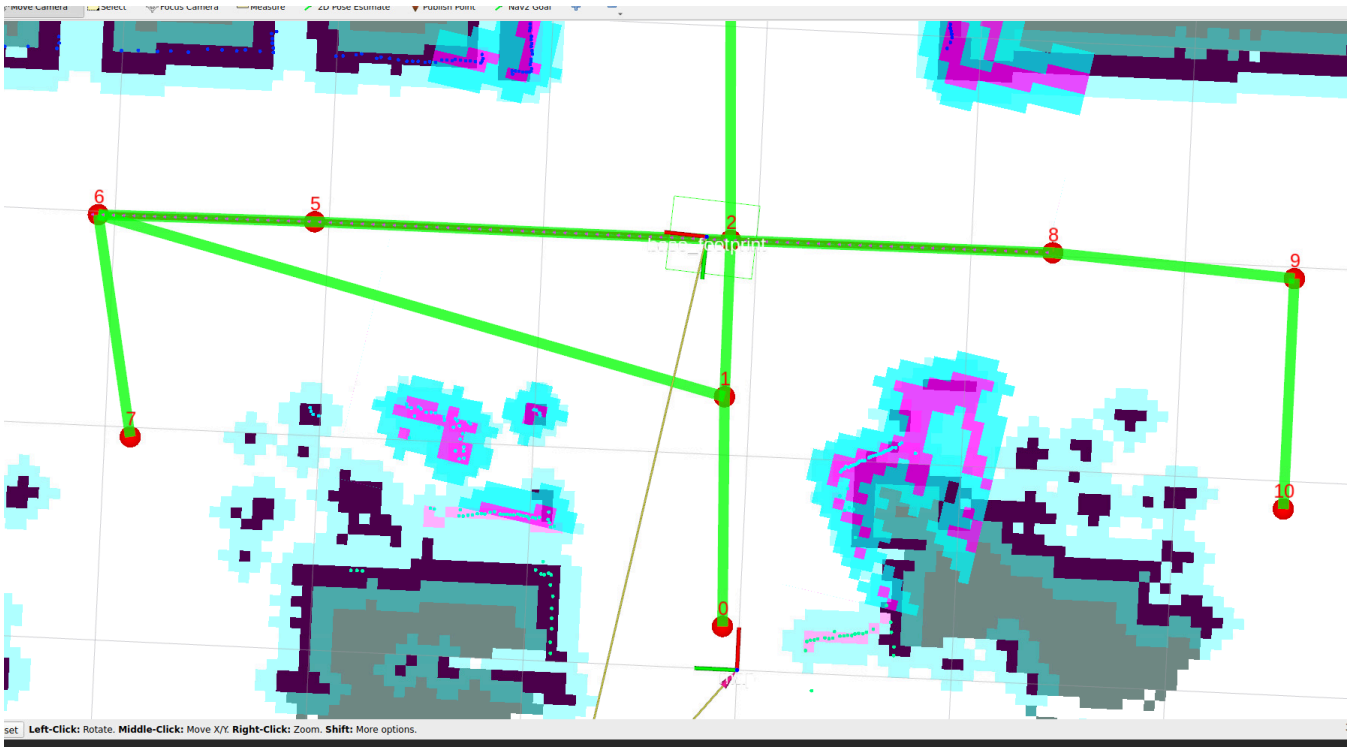
> **Tip**
>
> Optional launch parameter descriptions
>
> **map** The grid map file name to be loaded, default value is `map.yaml`
>
> **load_state_filename** The pose map file to be loaded. When using cartograph pure localization for global positioning, the pose map must be passed
>
> **roadnet_file** Road network description file, must be a complete file path
>
> **params_file** Navigation parameter file, default value is `os.path.join(get_package_share_directory('road_net_route'), 'config', 'nav2_kilted.yaml')`
>
> **localization_mode** Global positioning method, default is cartograph

Parameter Name	Input	Final Actual Path
`map`	map.yaml	`/root/map/map.yaml`
`pose_map`	map	`/root/map/map`
`roadnet_file`	**Absolute Path**	`/root/map/map.geojson`



- The test scenario here is: travel from waypoint 8 to waypoint 6 along the road network, then place obstacles on the road network planned path to trigger the replanning strategy

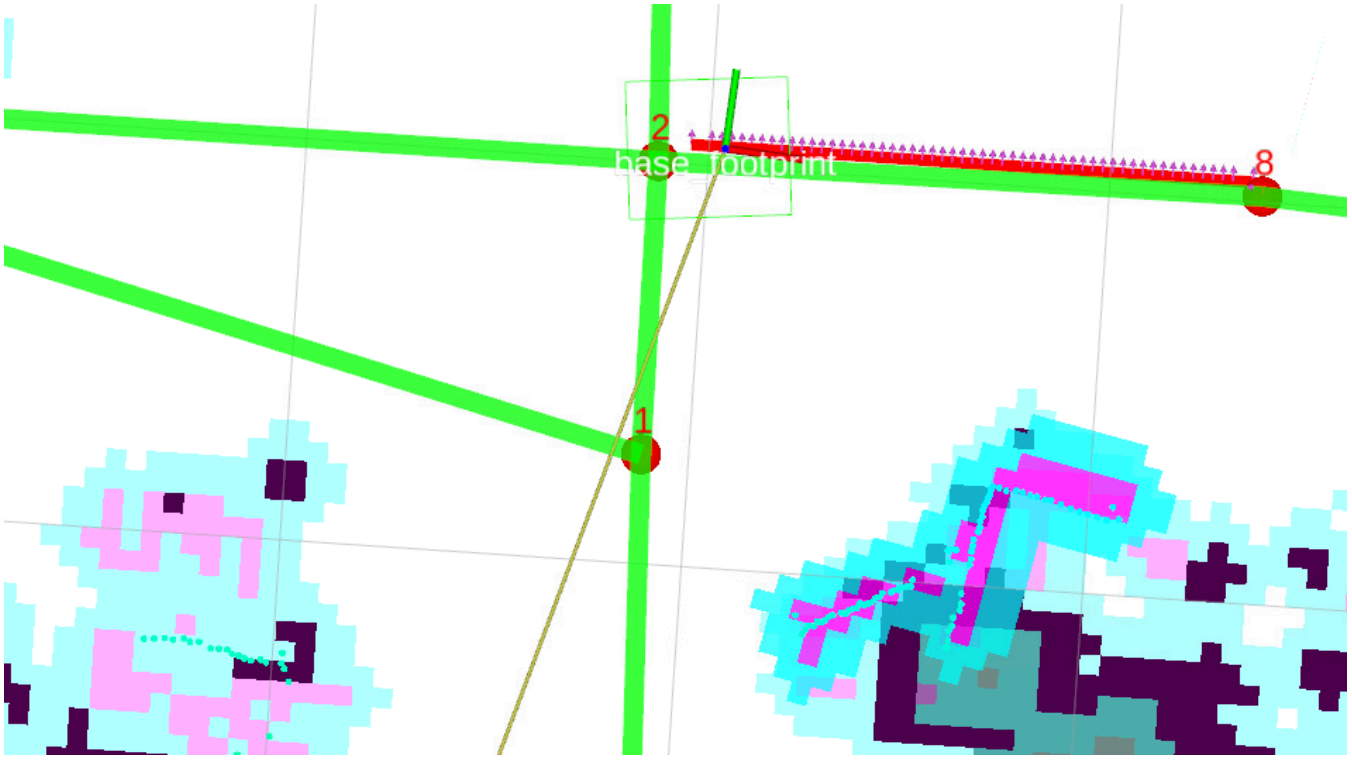
```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [8, 6]}"
```



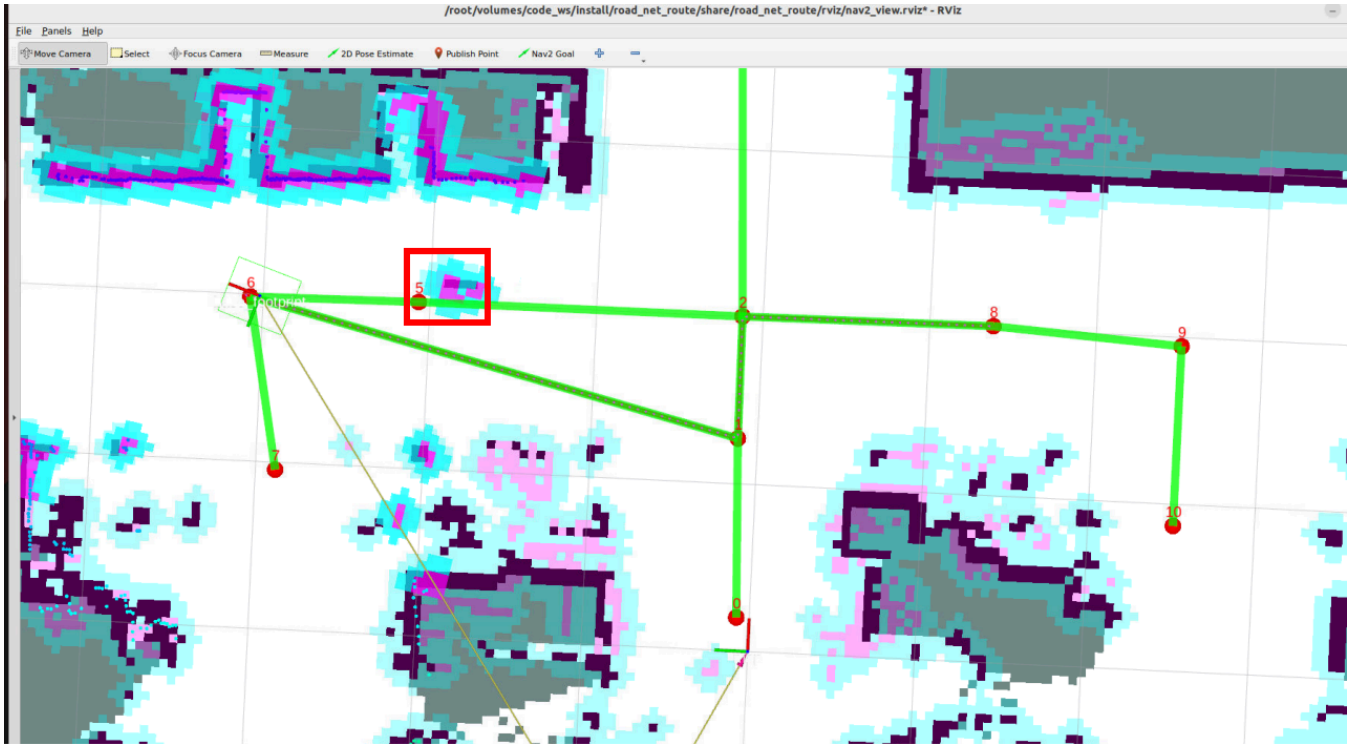
- Here you can see the terminal will prompt that obstacles exist on the path, then trigger replanning

```
[route_bridge-4] [INFO] [1764248055.001985898] [route_bridge]: id:2 ------> id:5 edge:16
[component_container_isolated-1] [INFO] [1764248055.687587193] [global_costmap.global_costmap]: StaticLayer: Resiz
ing costmap to 225 X 599 at 0.050000 m/pix
[route_bridge-4] [INFO] [1764248057.758462756] [route_bridge]: Path contains 7 potential obstacles.
[route_bridge-4] [INFO] [1764248058.249064867] [route_bridge]: Navigating to goal: 2.0027759075164795 0.1238360032
4392319...
```

- When obstacles are detected on the path and the path is not passable, it retreats to the last passed waypoint 8



- Then retry planning a drivable path from waypoint 8, and finally reach the target waypoint 6



3. Query Obstacle Cost Threshold

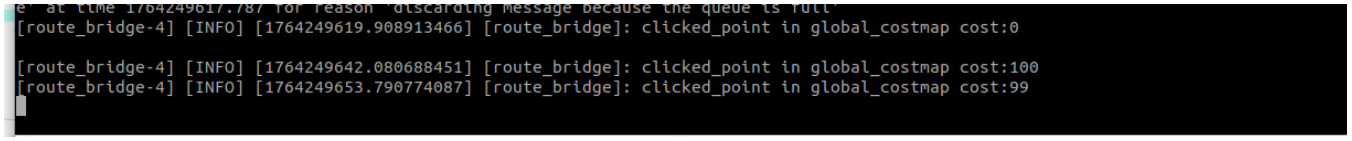
- Path obstacle detection here is determined by checking the cost values on the global path and the number of high-cost value points in the global path. The threshold for triggering replanning can be set by yourself

> **Note**
>
> - The obstacle detection threshold can generally use the factory default!!! If there are special scenario requirements, you can refer to this part of the tutorial to adjust the threshold for triggering replanning
> - Cost values are values used in the cost map to describe the occupied range of obstacles. The closer to the obstacle, the higher the cost value.

3.1 View Obstacle Cost Values

- Click the Publish Point tool, then left-click on the map where you want to view the map cost value to view the cost value of the specified location

- You can see that places without obstacles have a cost value of 0, and the center of obstacles has a cost value of 99-100



3.2 Modify Replanning Trigger Threshold

- Configuration file path

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted.yaml
```

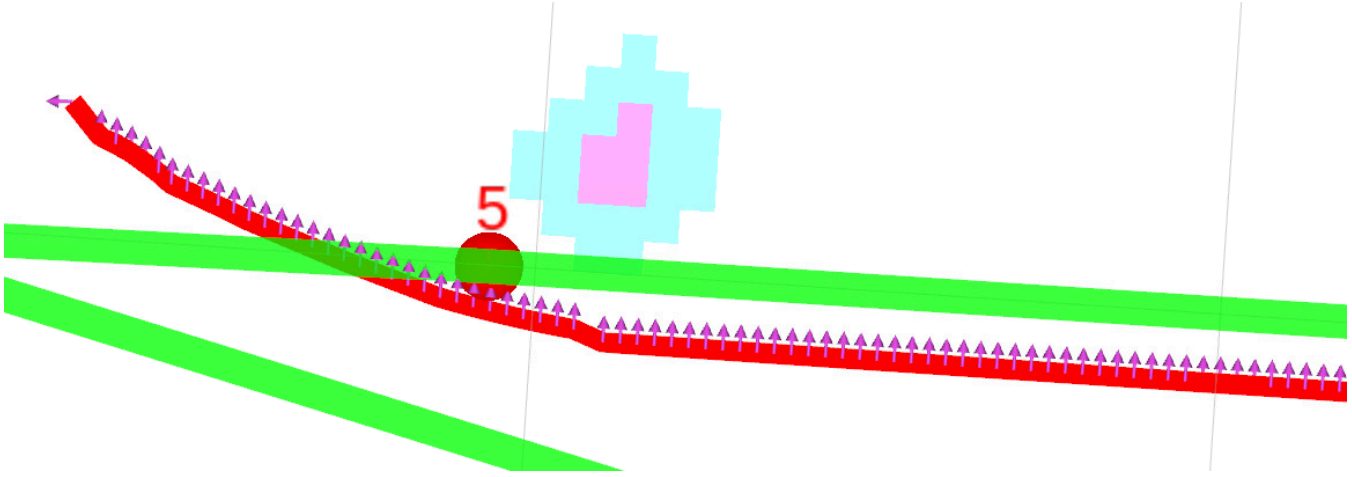
- Find the route_bridge node configuration parameters in the configuration file, where `COST_THRESHOLD` and `PATH_OBSTACLES_THRESHOLD` together determine the trigger threshold

```yaml
route_bridge:
  ros__parameters:
    roadnet_file: '/root/map/map.geojson'
    COST_THRESHOLD: 60
    replace2id: True
    PATH_OBSTACLES_THRESHOLD: 5
    max_consecutive_obstacles: 5
```

- Specific obstacle detection logic is as follows
- The global path planned by the road network function is of the ROS `nav_msgs/msg/Path` message type, which is a path composed of multiple pose points. During road network navigation, it checks whether all pose points on the global path have high-cost value points in the global cost map.

- If it exceeds the **COST_THRESHOLD** threshold, mark that point on the path as a high-cost value pose point
- Finally calculate how many pose points on the global are cost value pose points
- If the total number of high-cost value pose points is greater than **PATH_OBSTACLES_THRESHOLD**, the path is considered not passable and needs replanning
- The **max_consecutive_obstacles** parameter specifies the minimum number of consecutive detections required to identify something as an obstacle, preventing false positives.

> **Tip**
>
> If obstacle detection is not sensitive enough, you can reduce **COST_THRESHOLD** and **PATH_OBSTACLES_THRESHOLD**. If it's too sensitive, you can reduce these two values or adjust the obstacle inflation radius (for modifying obstacle inflation radius, refer to 04-Road Network Planning Navigation, 4.1 Navigation Controller Debugging)



4. Source Code Analysis

- Implementation program source code path:

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

4.1 Obstacle Detection and Replanning Strategy

- Iterate through each pose point of the global path, check if there are obstacles exceeding the threshold on the path through the global cost map, and return a boolean value indicating whether the path is blocked by obstacles.

- **World coordinates to map coordinates**: Call the cost map's `worldToMapValidated` method to convert world coordinates `(wx, wy)` to the cost map's **grid coordinates** `(mx, my)` (the cost map is a 2D grid map, each grid corresponds to a cost value).
- **Coordinate validity check**: If the converted grid coordinates are `None`, it means this path point is outside the global cost map range, print a warning log and skip this point.
- **Parameters**: `path:Path` → ROS2's `nav_msgs/msg/Path` message type, stores the robot's global planned path, containing a series of poses (`poses`).
- **Return value**: `bool` → `True` means the number of obstacles on the path exceeds the threshold, `False` means the path is passable.

```
def __check_path_obstacles(self, path:Path)->bool:
    """Check for obstacles on the global path."""
    pose_array=[]
    for pose in path.poses:
        wx=pose.pose.position.x
        wy=pose.pose.position.y

        mx, my = self.global_costmap.worldToMapValidated(wx,wy )
        if mx is None or my is None:
            self.get_logger().warn(f"Path point ({wx}, {wy}) is outside global costmap")
            continue

        cost = self.global_costmap.getCostXY(int(mx),int(my))

        if cost > self.COST_THRESHOLD:
            pose_array.append(pose)
    if len(pose_array) >self.PATH_OBSTACLES_THRESHOLD:
        self.get_logger().info(f"Path contains {len(pose_array)} potential obstacles.")
        return True
    return False
```

- When obstacles are detected on the global path, make the robot **retreat to the previous waypoint** and replan the path.

```
def __reroute_strategy_1(self,last_id):
    '''Replanning Strategy 1: Go back to the previous waypoint and replan the drivable
path.'''
    navigator= BasicNavigator()
    goal_x,goal_y=self.__get_node_coordinates(last_id)
    if goal_x is not None and goal_y is not None:
        goal_pose = PoseStamped()
        goal_pose.header.frame_id = "map"
        goal_pose.pose.position.x = goal_x
        goal_pose.pose.position.y = goal_y
        back_lastid = navigator.goToPose(goal_pose)
        while not navigator.isTaskComplete(task=back_lastid):
            time.sleep(0.1)
        if navigator.getResult() == TaskResult.SUCCEEDED:
```

```
            self.get_logger().info("back to last id  success.")
        else:
            self.get_logger().info("back to last id  failed.")
```

- handle_id_navigation and handle_pose_navigation are the core code responsible for road network planning navigation. The following parts in these two functions are the logic for detecting paths and triggering replanning

  When the path is detected as unfeasible, call **Replanning Strategy 1**: Make the robot **retreat to the previous waypoint** (coordinates corresponding to `last_node_id`). The core logic of this method has been explained before: initialize navigator → get coordinates of `last_node_id` → send navigation task to "retreat to this waypoint" → wait for task completion and print results. Move the robot from the current obstacle area back to a **safe predefined waypoint**, providing a stable starting point for subsequent path replanning.

  Complete process is as follows:

  1. Robot normally executes path tracking, continuously gets navigation feedback `feedback`.
  2. Check if there are obstacles on the path in `feedback`:
     - If no obstacles: continue executing the original path, no processing needed.
     - If there are obstacles: trigger replanning logic.
  3. Robot retreats to the previous safe waypoint (`last_node_id`).
  4. Use this waypoint as the starting point to replan the path to the target waypoint (`goal_id`).
  5. Get feedback of the new path, make the robot track the new path and continue moving.

```
if feedback is not None and self.__check_path_obstacles(feedback.path):
    self.__reroute_strategy_1(last_node_id)
    route_task = self.navigator.getAndTrackRoute(last_node_id, goal_id)
    feedback = self.navigator.getFeedback(task=route_task)
    follow_path_task = self.navigator.followPath(feedback.path)
```

```
    def handle_id_navigation(self, start_id:int, goal_id:int):
        '''Road network ID navigation'''

        route_task = self.navigator.getAndTrackRoute(start_id, goal_id)
        feedback=None
        follow_path_task=RunningTask.NONE
        last_feedback = None
        last_node_id =999
        while not self.navigator.isTaskComplete(task=route_task):
            feedback = self.navigator.getFeedback(task=route_task)
            if feedback is not None and self.__check_path_obstacles(feedback.path):
                self.__reroute_strategy_1(last_node_id)
                route_task = self.navigator.getAndTrackRoute(last_node_id, goal_id)
```

```
                feedback = self.navigator.getFeedback(task=route_task)
                follow_path_task = self.navigator.followPath(feedback.path)
            while feedback is not None:
                if  last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id != last_feedback.next_node_id):
                    self.get_logger().info(f'id:{feedback.last_node_id} ------> id:
{feedback.next_node_id} edge:{feedback.current_edge_id}')
                last_feedback = feedback
                last_node_id=feedback.last_node_id
                if feedback.rerouted :
                    self.get_logger().info('reroute a new path to follow!')
                    follow_path_task = self.navigator.followPath(feedback.path)
                feedback = self.navigator.getFeedback(task=route_task)

            if self.navigator.isTaskComplete(task=follow_path_task):
                print(follow_path_task)
                self.get_logger().info('Controller completed its task!')
                if follow_path_task != RunningTask.NONE :
                    self.navigator.cancelTask()

        result = self.navigator.getResult()
        if result == TaskResult.SUCCEEDED:
            self.get_logger().info(Fore.GREEN+"navigation task completed."+Fore.RESET)
            self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
        elif result == TaskResult.CANCELED:
            self.get_logger().warn("Navigation task canceled.")
        elif result == TaskResult.FAILED:
            self.get_logger().error("Navigation task failed.")
            self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

    def handle_pose_navigation(self, goal_pose:PoseStamped):
        '''Arbitrary pose navigation'''
        transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time(),rclpy.duration.Duration(seconds=5.0))
        if transform is None:
            self.get_logger().info(Fore.RED+"Failed to get TF transform "+Fore.RESET)
            return

        else:
            start_pose = PoseStamped()
            start_pose.header = transform.header
            start_pose.pose.position.x = transform.transform.translation.x
            start_pose.pose.position.y = transform.transform.translation.y
            start_pose.pose.orientation = transform.transform.rotation
            self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}"+Fore.RESET)
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(goal_pose.pose.position.x, goal_pose.pose.position.y)
            if self.replace2id:
                self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"
```

```
                                    f"\nnearest_goal_node_id:{nearest_goal_node_id},
distance:{nearest_goal_distance}"+Fore.RESET)
                route_task =
self.navigator.getAndTrackRoute(start=nearest_start_node_id,goal=nearest_goal_node_id)
            else:
                route_task =
self.navigator.getAndTrackRoute(start=start_pose,goal=goal_pose,use_start=True)

            feedback=None
            follow_path_task=RunningTask.NONE
            last_feedback = None
            last_node_id =999
            while not self.navigator.isTaskComplete(task=route_task):
                feedback = self.navigator.getFeedback(task=route_task)
                if feedback is not None and self.__check_path_obstacles(feedback.path):
                    self.__reroute_strategy_1(last_node_id)
                    route_task = self.navigator.getAndTrackRoute(last_node_id,
nearest_goal_node_id)
                    feedback = self.navigator.getFeedback(task=route_task)
                    follow_path_task = self.navigator.followPath(feedback.path)
                while feedback is not None:
                    if  last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id != last_feedback.next_node_id):
                        self.get_logger().info(f'id:{feedback.last_node_id} ------> id:
{feedback.next_node_id} edge:{feedback.current_edge_id}')
                    last_feedback = feedback
                    last_node_id=feedback.last_node_id
                    if feedback.rerouted :
                        self.get_logger().info('reroute a new path to follow!')
                        follow_path_task = self.navigator.followPath(feedback.path)
                    feedback = self.navigator.getFeedback(task=route_task)

                if self.navigator.isTaskComplete(task=follow_path_task):
                    self.get_logger().info('Controller completed its task!')
                    if follow_path_task != RunningTask.NONE :
                        self.navigator.cancelTask()

            result = self.navigator.getResult()
            if result == TaskResult.SUCCEEDED:
                if self.__adjust_pose(goal_pose):
                    self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)
                    self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))

            elif result == TaskResult.CANCELED:
                self.get_logger().info(Fore.YELLOW+"navigation task canceled."+Fore.RESET)

            elif result == TaskResult.FAILED:
                self.get_logger().info(Fore.RED+"navigation task failed."+Fore.RESET)
                self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
```

4.2 Map Cost Value Query Implementation Method

- Call the global cost map's `worldToMapValidated` method to convert **world coordinates** `(x, y)` **to the cost map's grid coordinates** `(mx, my)`.
- The cost map is a **2D grid map** (each grid corresponds to a small area in the real world, 0.05m×0.05m), `worldToMapValidated` is a safe conversion method: if coordinates exceed the cost map range, it returns `None`, avoiding out-of-bounds errors caused by direct conversion.
- **Get cost value**: Call the cost map's `getCostXY` method, pass the integer form of grid coordinates (grid coordinates are discrete integers, avoid errors caused by passing floating-point types), to get the **cost value** `cost` of this grid.

```
    def get_costmap_value(self, msg:PointStamped):
        '''In RViz, you can click to view the cost of a point on the global cost map.'''
        x=msg.point.x
        y=msg.point.y
        mx, my = self.global_costmap.worldToMapValidated(x, y)
        if mx is None or my is None:
            self.get_logger().warn(f"Path point ({x}, {y}) is outside global costmap")
            return
        cost = self.global_costmap.getCostXY(int(mx),int(my))
        self.get_logger().info(f"clicked_point in global_costmap cost:{cost}")
```